

Home / AI & ML / GitHub Copilot

How to maximize GitHub Copilot's agentic capabilities

A senior engineer's guide to architecting and extending Copilot's real-world applications.



Ari LiVigni · @arilivigni

February 2, 2026

🕒 5 minutes

Share:



Modern engineering work rarely lives in a single file. Real systems evolve across years of incrementally layered decisions—some good, some accidental. A single feature request (“Add tagging to notes,” “Refactor the validation layer,” “Support a new consumer on our

API”) often touches controllers, domain models, repositories, migrations, tests, documentation, and deployment strategy.

Copilot’s agentic capabilities don’t replace your judgment in these situations—they amplify it. When used well, Copilot becomes a partner in system design, refactoring, modernization, and multi-file coordination.

This guide focuses on architecture-aware, multi-step workflows used every day by staff engineers, but written to be accessible for earlier-career engineers who want to understand *how* senior engineers think—and how Copilot can accelerate their own growth.

It draws on four [GitHub Skills exercises](#) (linked below), and builds toward a complete, real-world scenario: extending a small modular Notes Service with a tagging subsystem, refactoring a validation layer, designing a safe migration, and modernizing tests.

Before you start

You’ll get the most out of this guide if you have:

- GitHub Copilot with agent mode enabled
- Some familiarity with service-layer architectures (Node, Python, Go—language doesn’t matter)
- Copy a GitHub Skills exercise template to your handle or organization (use the green “Copy Exercise” button)
- A willingness to let Copilot propose solutions—and the judgment to inspect and challenge them

If you’re earlier in your career, don’t worry. Each section explains **why** these patterns matter and how to practice them safely.

Learn interactively at your own pace with GitHub Skills

The following free modules map directly to the workflows in this guide:

- [Expand your team with Copilot](#): multi-step agentic execution
- [Build applications with Copilot \(agent mode\)](#): task-driven code generation
- [Modernize your legacy codebases with Copilot](#): refactoring and migrations
- [Customize Your GitHub Copilot experience](#): Customize GitHub Copilot's behavior with custom instructions, prompts, and custom agents for your specific development workflows

Each one is designed to be forkable, inspectable, and safe for experimentation. You can also explore all our GitHub Skills courses below.

[Explore GitHub Skills >](#)

Using Copilot for system design and decomposition (not just scaffolding)

Senior engineers rarely begin by writing code. They begin by identifying boundaries: domain logic, data access, interfaces, and how modules should interact.

Copilot agent mode can help by revealing structural issues and proposing architectures.

Prompt:

```
Analyze this service and propose a modular decomposition with domain,
infrastructure, and interface layers.
```

```
Identify anti-patterns, coupling issues, and potential failure points.
```

You'll typically get back:

- Proposed module boundaries
- Cross-layer coupling concerns
- Async/transaction pitfalls

- Duplication or tight weaving of responsibilities
- Testability and observability implications

This transforms Copilot from an autocomplete tool into a design reviewer.

You can push further by asking it to compare architectures:

```
Compare hexagonal architecture vs. layered architecture for this codebase.
```

```
Recommend one based on the constraints here. Include tradeoffs.
```

Want to try it yourself? Use these proposals as starting points.

Building a modular service using agentic workflows

Once boundaries are defined, Copilot can coordinate changes across modules.

Prompt:

```
Implement the domain, controller, and repository layers as distinct modules.
```

```
Use dependency inversion to reduce coupling.
```

```
Document assumptions and contracts for each module.
```

Copilot will typically generate:

- Domain model interfaces
- Repository abstractions
- Controller logic calling domain services
- A short Markdown summary describing each module

For earlier-career engineers, this provides exposure to real engineering patterns. For senior engineers, it provides leverage and reduces boilerplate overhead.

Feature work with architectural awareness (example: tagging subsystem)

Adding a tagging subsystem is a deceptively simple request with meaningful architectural implications.

Even this single feature forces decisions across the system:

- **Data modeling:** embedded tags vs. normalized tables vs. many-to-many relationships
- **Search behavior:** how tags affect indexing, filtering, and relevance
- **API contracts:** whether tags are first-class resources or an implementation detail
- **Validation boundaries:** where constraints and invariants are enforced
- **Migration and rollout:** additive vs. breaking changes and rollback strategy

Before touching code, ask Copilot to map the impact.

Prompt:

Propose the architectural changes required to add a tagging subsystem.

Identify migration needs, cross-cutting concerns, caching or indexing implications, and potential regressions.

Copilot may identify:

- Tag-note relationships (one-to-many or many-to-many)
- Migration strategy
- Impact to search logic
- Required test updates
- Changes in validation logic
- Implications on external API consumers

This is the staff-level lens that Copilot can help junior developers adopt.

Then implement it:

Implement the tagging domain model, schema changes, repository updates, and controller logic.

Update tests and documentation. Show each change as a diff.

Example output (simplified)

Migration example:

```
ALTER TABLE notes ADD COLUMN tags TEXT DEFAULT '[]';
```

Domain model example:

```
export interface Tag {  
  id: string;  
  label: string;  
}  
  
export interface Note {  
  id: string;  
  title: string;  
  body: string;  
  tags: Tag[];  
}
```

Controller update (partial):

```
await noteService.addTag(noteId, { label: req.body.label });
```

This is where agent mode shines: coordinating multiple files with consistent intent.

Schema migrations and safe rollout strategies

At senior levels, the hardest part isn't writing SQL. It's designing a change that is:

- Backward compatible
- Reversible
- Safe under load
- Transparent to dependent systems

Ask Copilot to reason about this:

Prompt:

```
Generate an additive, backward-compatible schema migration to support the tagging subsystem.
```

```
Describe the rollback plan, compatibility window, and expected impact to existing clients.
```

This forces Copilot to consider:

- Mon-breaking additive fields
- Optional fields vs. required fields
- Whether a dual-read or dual-write strategy is needed
- Safe rollback procedures
- API versioning implications

If you're earlier in your career, this offers lessons on how safe migrations are designed. And if you're more experienced, this gives you a repeatable workflow for multi-step schema evolution.

Advanced refactoring with agentic workflows

Let's perform a real cross-module refactor: extracting validation out of controllers into a domain service.

Prompt:

```
Create a step-by-step refactor plan to extract validation logic into a domain service.
```

```
Identify affected modules and required test updates.
```

Copilot may output something like:

1. Introduce domain `validationService`
2. Move validation logic from controller to service
3. Update controllers to use new service
4. Update repository logic where validation assumptions leak
5. Update domain tests
6. Update integration tests

Execute in incremental steps

Prompt:

```
Execute steps 1-3 only. Stop before controller rewrites.
```

```
Provide detailed diffs and call out risky areas.
```

This is a low-blast-radius refactor, modeled directly in the IDE.

Modernizing test strategy

Instead of asking Copilot “write tests,” ask it to assess the entire suite.

Prompt:

```
Analyze the current test suite and identify systemic gaps.
```


Recommend a modernization plan including contract, integration, and domain-layer tests.

Then implement contract tests:

```
describe("NotesRepository contract", () => {  
  test("create + fetch returns a fully hydrated note object", async () => {  
    const note = await notesRepo.create({ title: "Test", body: "..." });  
    const fetched = await notesRepo.get(note.id);  
  
    expect(fetched).toMatchObject({ title: "Test" });  
    expect(fetched.id).toBeDefined();  
  });  
});
```

This elevates testing into an architectural concern.

A complete end-to-end workflow

Bringing it all together, here's a real sequence you might run with Copilot:

1. **Ask Copilot to analyze the existing architecture:** identify hazards, modularization opportunities
2. **Define module boundaries:** domain, repository, controller layers
3. **Add tagging subsystem:** architectural assessment to implementation to tests to doc updates
4. **Create a backward-compatible migration:** additive schema to rollback plan
5. **Perform a targeted refactor:** validation layer extraction
6. **Modernize tests:** contract + integration + domain tests

This workflow is architecturally realistic—and a model for how Copilot becomes a system-level collaborator.

What agent mode is *not* for

It's important to clarify that agent mode is not ideal for:

- Altering domain invariants without human review
- Redesigning cross-service ownership boundaries
- Replacing logic driven by institutional knowledge
- Large sweeping rewrites across hundreds of files
- Debugging deep runtime issues

Copilot should *support* your decision-making, not replace it.

Key takeaways

- Agent mode excels at **multi-step, multi-file engineering workflows**.
- Copilot is a **design and coordination partner**, not a replacement for judgment.
- Use Copilot for decomposition, refactoring, migrations, test strategy, and documentation.
- Start with architecture, not implementation.
- You can safely practice all these patterns in GitHub Skills.

[Learn more about agent mode >](#)

Where to go next

Here's where GitHub Skills comes in—not as “beginner content,” but as a set of guided, self-contained labs that reinforce the patterns above.

Even senior engineers will benefit: These exercises are structured so you can reliably recreate complex workflows and test Copilot's behavior in controlled environments.

[Explore GitHub Skills >](#)

Tags :

GitHub Copilot

GitHub Skills

Written by

**Ari LiVigni**[@arilivigni](#)

Senior Service Delivery Engineer, GitHub

More on GitHub Copilot

From pixels to characters: The engineering behind GitHub Copilot CLI's animated ASCII banner

Learn how GitHub built an accessible, multi-terminal-safe ASCII animation for the Copilot CLI using custom tooling, ANSI color roles, and advanced terminal engineering.

Aaron Winston

Power agentic workflows in your terminal with GitHub Copilot CLI

Explore the GitHub Copilot CLI and try interacting with Copilot directly from your terminal.

Dylan Birtolo